■ 3.7.8 Solving Linear Systems

Many calculations involve solving systems of linear equations. In many cases, you will find it convenient to write down the equations explicitly, and then solve them using `Solve`.

In some cases, however, you may prefer to convert the system of linear equations into a matrix equation, and then apply matrix manipulation operations to solve it. This approach is often useful when the system of equations arises as part of a general algorithm, and you do not know in advance how many variables will be involved.

A system of linear equations can be stated in matrix form as $\mathbf{M}.\mathbf{x} = \mathbf{b}$, where $\mathbf{x}$ is the vector of variables.

`LinearSolve[`*m*`, `*b*`]`	give the vector *x* which solves the matrix equation *m*`.`*x* `==` *b*
`NullSpace[`*m*`]`	a list of basis vectors whose linear combinations satisfy the matrix equation *m*`.`*x* `== 0`
`RowReduce[`*m*`]`	a simplified form of *m* obtained by making linear combinations of rows

Functions for solving linear systems.

Here is a 2 × 2 matrix.

```
In[1]:= m = {{1, 5}, {2, 1}}
Out[1]= {{1, 5}, {2, 1}}
```

This gives two linear equations.

```
In[2]:= m . {x, y} == {a, b}
Out[2]= {x + 5 y, 2 x + y} == {a, b}
```

You can use `Solve` directly to solve these equations.

```
In[3]:= Solve[ %, {x, y} ]
                -a   5 b        2 a   b
Out[3]= {{x -> -- + ---, y -> --- - -}}
                9     9          9    9
```

You can also get the vector of solutions by calling `LinearSolve`. The result is equivalent to the one you get from `Solve`.

```
In[4]:= LinearSolve[m, {a, b}]
             a   5 b     -2 a   b
Out[4]= {-(- - ---), -(---- + -)}
             9    9       9     9
```

Another way to solve the equations is to invert the matrix `m`, and then multiply `{a, b}` by the inverse. This is not as efficient as using `LinearSolve`.

```
In[5]:= Inverse[m] . {a, b}
         -a   5 b  2 a   b
Out[5]= {-- + ---, --- - -}
         9     9    9    9
```

Particularly when you have large, sparse, matrices, `LinearSolve` is the most efficient method to use.

If you have a square matrix $\mathbf{m}$ with a nonzero determinant, then you can always find a unique solution to the matrix equation $\mathbf{m}.\mathbf{x} = \mathbf{b}$ for any $\mathbf{b}$. If, however, the matrix $\mathbf{m}$ has determinant zero, then there can be no vector $\mathbf{x}$ which satisfies $\mathbf{m}.\mathbf{x} = \mathbf{b}$ for a particular $\mathbf{b}$. This situation corresponds to the case in which the linear equations embodied in $\mathbf{m}$ are not independent.

When $\mathbf{m}$ has determinant zero, it is nevertheless always possible to find vectors $\mathbf{x}$ that satisfy $\mathbf{m}.\mathbf{x} = \mathbf{0}$. The set of vectors $\mathbf{x}$ satisfying this equation form the *null space* or *kernel* of the matrix $\mathbf{m}$. Any of the vectors can be expressed as a linear combination of a particular set of basis vectors, which can be obtained using `NullSpace[`*m*`]`.

Here is a simple matrix, corresponding to two identical linear equations.

```
In[6]:= m = {{1, 2}, {1, 2}}
Out[6]= {{1, 2}, {1, 2}}
```

The matrix has determinant zero.

```
In[7]:= Det[ m ]
Out[7]= 0
```

`LinearSolve` cannot find a solution to the equation $\mathbf{m}.\mathbf{x} = \mathbf{b}$ in this case.

```
In[8]:= LinearSolve[m, {a, b}]
LinearSolve::nsl:
    Linear equation encountered which has no solution.
Out[8]= LinearSolve[{{1, 2}, {1, 2}}, {a, b}]
```

There is a single basis vector for the null space of `m`.

```
In[9]:= NullSpace[ m ]
Out[9]= {{-2, 1}}
```

Multiplying the basis vector for the null space by `m` gives the zero vector.

```
In[10]:= m . %[[1]]
Out[10]= {0, 0}
```

Here is a simple symbolic matrix with determinant zero.

```
In[11]:= m = {{a, b, c}, {2 a, 2 b, 2 c}, {3 a, 3 b, 3 c}}
Out[11]= {{a, b, c}, {2 a, 2 b, 2 c}, {3 a, 3 b, 3 c}}
```

The basis for the null space of `m` contains two vectors. Any linear combination of these vectors gives zero when multiplied by `m`.

```
In[12]:= NullSpace[ m ]
               c                 b
Out[12]= {{-(-), 0, 1}, {-(-), 1, 0}}
               a                 a
```

An important feature of `LinearSolve` and `NullSpace` is that they work with *rectangular*, as well as *square*, matrices.

When you represent a system of linear equations by a matrix equation of the form $\mathbf{m}.\mathbf{x} = \mathbf{b}$, the number of columns in $\mathbf{m}$ gives the number of variables, and the number of rows give the number of equations. There are a number of cases.

Underdetermined	number of independent equations less than the number of variables; many solutions may exist
Overdetermined	number of independent equations more than the number of variables; solutions may not exist
Nonsingular	number of independent equations equal to the number of variables, and determinant nonzero; a unique solution exists
Consistent	at least one solution exists
Inconsistent	no solutions exist

Classes of linear systems represented by rectangular matrices.

This asks for the solution to the inconsistent set of equations $x = 1$ and $x = 0$.

```
In[13]:= LinearSolve[{{1}, {1}}, {1, 0}]

LinearSolve::nsl:
    Linear equation encountered which has no solution.

Out[13]= LinearSolve[{{1}, {1}}, {1, 0}]
```

This matrix represents two equations, for three variables.

```
In[14]:= m = {{1, 3, 4}, {2, 1, 3}}

Out[14]= {{1, 3, 4}, {2, 1, 3}}
```

`LinearSolve` gives one of the possible solutions to this underdetermined set of equations.

```
In[15]:= v = LinearSolve[m, {1, 1}]

          2  1
Out[15]= {-, -, 0}
          5  5
```

When a matrix represents an underdetermined system of equations, the matrix has a non-trivial null space. In this case, the null space is spanned by a single vector.

```
In[16]:= NullSpace[m]

Out[16]= {{-1, -1, 1}}
```

If you take the solution you get from `LinearSolve`, and add any linear combination of the basis vectors for the null space, you still get a solution.

```
In[17]:= m . (v + 4 %[[1]])

Out[17]= {1, 1}
```

You can find out the number of redundant equations corresponding to a particular matrix by evaluating `Length[NullSpace[`*m*`]]`. Subtracting this quantity from the number of columns in m gives the *rank* of the matrix m.